

A Mini Project Report on

“AI POWERED MULTILINGUAL CYBERBULLYING DETECTOR”

**Submitted in partial fulfillment of the requirement for
Degree in Bachelor of Technology (Information Technology)**

By

Sania Musliar (5024139)
Navya Roshni (5024142)
Vishakha Talele (5024163)
Tanaya Deshmukh (5024168)

Guided by:

Dr. Trupti Lotlikar



Department of Information Technology
Fr. Conceicao Rodrigues Institute of Technology
(An Autonomous Institute & Permanently Affiliated to University of Mumbai)
Sector 9A, Vashi, Navi Mumbai – 400703

University of Mumbai
2025-2026

CERTIFICATE

This is to certify that the mini project entitled

AI POWERED MULTILINGUAL CYBERBULLYING DETECTOR

Submitted By

Sania Musliar
Navya Roshni
Vishakha Talele
Tanaya Deshmukh

In partial fulfilment of degree of **B.Tech. in Information Technology** for term work of the
Semester IV – Mini Project- 1B is approved.

External Examiner

Internal Examiner

Internal Guide

Head of the Department

Principal

Date: -

College Seal

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that We have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
(Sania Musliar - 5024139)

(Signature)
(Navya Roshni - 5024142)

(Signature)
(Vishakha Talele - 5024163)

(Signature)
(Tanaya Deshmukh - 5024168)

Date:

ABSTRACT

The rise of social media and online communication has resulted in a growing increase in cyberbullying and toxic behavior, creating unsafe digital spaces and negatively affecting users' mental health. To address this challenge, this project develops an AI-powered browser extension capable of real-time multilingual toxicity detection, aimed at promoting safer and more inclusive online interactions.

The system employs Natural Language Processing (NLP) and Machine Learning techniques to analyze online text and identify harmful content such as hate speech, harassment, and abusive language. It supports multiple languages, enabling users from different linguistic backgrounds to benefit from effective toxicity detection. The browser extension operates directly within web platforms, offering instant detection and alerts while maintaining user privacy and lightweight performance. Testing and evaluation indicate that the model performs efficiently, providing accurate results with minimal processing time. The system can be easily integrated into everyday browsing environments, making it accessible and practical for general users.

Overall, this project contributes to building a safer and more inclusive digital environment by using AI to detect and reduce toxic communication across languages and platforms in real time.

The system further enhances detection by classifying toxicity into multiple levels (low, medium, high) and provides user-controlled interaction through a "View" button to reveal hidden content.

INDEX

Sr. No.	Topic	Page No.
1	Introduction 1.1 Back ground 1.2 Motivation 1.3 Problem Definition 1.4 Scope / Assumptions 1.5 Issues / Limitations	9-13
2	Literature Survey and Analysis 2.1 Related work 2.2 Existing System 2.3 Requirements Analysis	14-20
3	System Design 3.1 Architectural Diagram 3.2 Flow chart 3.3 ER Diagram for database 3.4 DFD 3.5 Execution flow	21-30
4	Implementation Details 4.1 Software Requirements 4.2 Solution Approach / Methodology 4.3 Algorithm and Core Functionality 4.4 Outputs and Results	31-40
5	Software testing/test cases 5.1 Unit Testing 5.2 Integration Testing 5.3 Model Evaluation 5.4 Real Time Testing 5.5 Usability Training	41-44
6	Experimental Results 6.1 GUI 6.2 Gantt Chart	45-49

7	Conclusion and Future Scope 7.1 Conclusion 7.2 Future Scope	50-52
6	References	53-54
7	Acknowledgements	55
8	Appendix A: Code Sample	56-57
	Appendix B: i. Project competition participation/winner's certificates/Certificates of papers presented at conferences/ Papers published in conference/journal	58-60
	ii. Plagiarism report (summary) of mini project report	
	Appendix C: Mapping to Sustainable Development Goals (SDGs), Mapping to POs and PSOs	

LIST OF FIGURES

Sr. No.	Name of the Figure	Page No.
3.1	Architectural Diagram / Block Diagram	22
3.2	System Flow Chart (Execution Flow / Working of System)	25
3.3	ER diagram for database	26
3.4	DFD Diagram (Level 1)	27
3.5	Execution Flow	28
4.1	Toxic comment filtering on Instagram	39
4.2	Before toxic comment filtering on Youtube	40
4.3	After toxic comment filtering on Youtube	40
4.4	Before toxic comment filtering on X	40
4.5	After toxic comment filtering on X	40
5.1	Actual vs Predicted Toxicity Distribution	43
6.1	Admin login interface	46
6.2	Admin dashboard showing toxic comments	47
6.3	Admin Dashboard Showing Stored Comments and Predictions	47
6.4	Timeline chart	48

LIST OF TABLES

Sr. No.	Name of the Figure	Page No.
2.1	Review of Related Work	16-18
2.2	Requirements	20
4.1	System Architecture: Key Modules and Technologies	34
4.2	Mitigation Algorithm Explanation	39

CHAPTER 1

INTRODUCTION

1.Introduction

1.1 Background:

The digital world has radically altered the way human beings communicate, exchange information and air their views. Online forums, messaging apps, and social media are now a resource that has become a crucial aspect of daily communication. Their user base is in billions and reaches all parts of the world, enabling instant connectivity, cultural exchange and sharing of knowledge. Nonetheless, a worrying escalation of negative online behaviours that includes cyberbullying, harassment, and hate speech also accompany this connectivity.

Abuse words, personal attacks, and discriminating remarks are all toxic forms of online communication, which have become an increasing problem. This malicious information does not only interfere with healthy communication, but it also has an impact on the mental well-being of the users. Online toxicity can cause anxiety and depression, as well as emotional distress, which results in lowered self-esteem and in other instances, social withdrawal.

Online toxicity is not a problem of one specific platform or location; it is a worldwide problem that cuts across language, culture, and demographics. With the multilingualization of digital spaces, the traditional forms of moderation based on manual reporting or on the filtering of keywords remain out of step. Human language is very complex, which involves sarcasm, slang, and cultural differences, therefore, it becomes even harder to identify the toxic content.

In this regard, Artificial Intelligence (AI) and Natural Language Processing (NLP) have become potent resources that can analyse the text and detect harmful information more efficiently. These technologies allow systems to process context, sentiment and intent beyond basic keyword spotting. By implementing toxicity detection systems based on AI into the digital world, it is possible to make the online world a safer and more inclusive place.

1.2 Motivation

The biggest reason why this project is necessary is to tackle the mental health issues and emotional damage related to being exposed to online toxicity. Research and international surveys have shown a direct relationship between exposure to hate speech or cyberbullying and higher rates of anxiety, stress and depression among users, especially in teenagers and young adults.

Current moderation systems usually do not have sufficient capacity to handle this increasing issue. Some of them use reports provided by users or preset keywords that are not able to represent the richness of languages, phrases, and situations where toxicity takes place. Moreover, these

systems are usually do not have real-time detection services, which implies that dangerous content usually arrives to users prior to being detected or removed.

The language barrier is another major weakness. The majority of toxicity detection models are pre-trained on English or a small number of major languages, and thus are not useful in identifying harmful content in regional or mixed-language messages (like Hinglish or Spanglish). This is a major limitation in a world where people are interconnected with other languages spoken in various countries, and conversations are usually mixed.

The motivation, hence, is to devise an AI-based multilingual system of toxicity detection that could work in real time and help users by providing assistance immediately. By creating a browser extension, the project will enable users with a tool that will identify, mark and control toxic content before it can influence them psychologically.

1.3 Problem Definition:

Although machine learning and language processing have advanced technology, the existing toxicity detection systems still lack effectiveness in a variety of critical aspects. The most frequent problems observed in the existing systems are:

1. Language Restriction:

The majority of the models are trained on English or a small set of languages. They do not read mixed-language texts or regional differences, which leads to wrong or incorrect detections in a multilingual setting.

2. The absence of Real-Time Intervention:

Most systems do not identify any toxic material until after it has been posted or reported. Such delays enable the spread of harmful messages to audiences and result in psychological or reputational harm before the process of moderation occurs.

3. Limited Automation:

Traditional systems usually rely on the manual reporting by the user or human moderator, which is time consuming and subject to bias. Automated tools that are able to detect and act on toxicity are required and must be under user control.

With these constraints in mind, the issue that this project aims to solve is the lack of a real-time, multilingual, AI-driven toxicity detecting tool, which, in turn, can identify, classify, and prevent harmful behaviour on the internet.

The proposed system tries to address this gap by creating a browser extension that is based on AI and NLP models. It will identify poisonous language in various platforms including social media,

chat rooms and comment boxes in real time. It will also give feedback to the users to enable them to control or filter dangerous materials prior to viewing.

This strategy will guarantee upstream action rather than downstream control. This will allow the system to identify the toxic language with high accuracy and speed without invasion of privacy of the user because of the multilingual NLP methods and effective AI classification models. The project also focuses on ethical design of AI, whereby detection mechanisms do not infringe on the freedom of speech and at the same time, will not interfere with user safety and emotional well-being.

The traditional systems do not have any facility to analyze the severity and give control to the user as most existing systems only do binary classification (toxic/non-toxic).

1.4 Scope/Assumption

The context of this project is the creation of a browser-based toxicity detection extension that makes use of Artificial Intelligence (AI) and Natural Language Processing (NLP) to detect dangerous or offensive content on the Web in real-time. The system is intended to process mainly text-based data, and it is aimed at identifying hate speech, harassment, profanity and other types of toxic communication in a wide variety of languages, such as code-mixed, and regional texts.

The project will improve the digital safety and mental health by ensuring that users receive a warning before being exposed to toxic content. It can be used in social media, on-line communities, and through web-based chat interfaces. The tool will identify the toxic phrases or words and will enable the user to do the necessary actions, which include ignoring, filtering or reporting of the content.

In this project, it is assumed that the users will be able to maintain a stable internet connection and have access to a modern web browser that can support the extension. It also presupposes that the AI model will be trained on a trustworthy and balanced multilingual dataset, so as to be equally detected across languages and dialects. The study also presupposes responsible use of the tool in question - in other words, the users will utilize it as an aid to the creation of a healthy online environment and not as a form of censorship or control.

1.5 Issues/Limitation

Despite the intention to create an efficient and solid system of online toxicity detection, some problems and limitations are identified in this project:

- Complexity of language: Multilingual support might not be able to understand slang, sarcasm or mixed language phrases that differ among cultures.
- Poor Contextual & Sentimental Analysis: Due to the absence of in-depth contextual and sentiment analysis in the project, the system may sometimes interpret neutral or humorous statements as toxic.
- Reliance on Training Data: The quality of the dataset and the variety of data are crucial in the accuracy of the detection. Performance may be affected by any bias in the data.
- Performance Constraints: Detection in real time on the browser might be constrained by system memory, processing speed and internet latency.

CHAPTER 2.
LITERATURE SURVEY & ANALYSIS

2 Literature Survey and Analysis

1. Cyberbullying Detection of Resource-Constrained Language from social media using Transformer-Based Approach [year 2024]

Authors: Sihab-Us-Sakib, Md. Ashraful Islam, S. Hossain

The study aimed at creating an automated model of detecting cyberbullying in Bengali, a low-resource language [1]. The authors developed a new dataset, the Cyberbullying Bengali Dataset (also known as CBD) of 2,751 manually labeled Facebook, YouTube, and Instagram comments [1]. Several machine learning and deep learning algorithms were tested including SVM, Naïve Bayes, Random Forest, CNN, GRU, and BiLSTM [1]. Transformer-based models such as mBERT, BanglaBERT and XLM-RoBERTa were also used, with the XLM-RoBERTa having the best F1-score of 0.83 and an accuracy of 82.61% [1]. This research demonstrated that the transformer based architectures are more efficient than the conventional ones in classifying abusive text despite the resource limitations of the language, but only Bengali datasets could be used [1].

2. BullStop: A Mobile App for Cyberbullying Prevention [year 2020]

Authors: Semiu Salawu, Yulan He, Jo Lumsden

BullStop is a lifeline mobile app which aims at identifying and preventing cyberbullying and online abuse in social media platforms [2]. It also uses deep learning, including BERT, DistilBERT, RoBERTa, and XLNet, which are incorporated into a microarchitecture to scale [2]. The app is linked through APIs (e.g., Twitter) to track posts, identify messages related to bullying and automatically delete offensive posts or block attackers [2]. It has an automatic learning process that enables it to improve itself over time with user feedback [2]. The research proved the practicality of implementing AI-driven detection in the actual mobile environment but only with English data [2].

3. Intelligent Multi-Lingual Cyber-Hate Detection in Online Social Networks [year 2023]

Author: Donia Gamal, Mahmoud Hamed, Ahmed Said

In this paper, a multilingual cyber-hate detection systems taxonomy and analysis was outlined [3]. It has classified various forms of hate on the internet like exclusion, denigration, cyberstalking, trolling, and harassment [3]. The authors have examined over 25 multilingual sets of data on platforms, including Twitter, Facebook, WhatsApp, and YouTube, and tested algorithms, including Naive Bayes, SVM, LSTM, and BERT-based models in different languages [3]. The paper has highlighted the role of multilingual transformer models (such as

XLM-RoBERTa and mBERT) in enhancing the detection of hate speech across the world [3]. Nevertheless, it was an abstract survey that lacked a model that was put into practice [3].

4. Intelligent Multi-Lingual Cyber-Hate Detection in Online Social Networks [year 2025]

Author: Bishal Shyam Purkayastha, Rupak Dey, Aritra Paul

The presented work proposed a hybrid ML-DL system that implements the Random Forest, Extra Trees, AdaBoost, BiLSTM, and BERT models to detect cyberbullying [4]. Two real-world datasets Mandalay and Kaggle were resampled by random oversampling [4]. The BERT model was also able to reach an accuracy of 92 percent and perform better in context understanding [4]. The study noted the possibility of hybrid architectures in enhancing accuracy and generalization. Nevertheless, the system was not multilingual scalable and only worked with English text [4].

Table 2.1: Review of Related Work

Sr. No	Paper	Authors	Description	Advantages	Limitations
1	Cyberbullying Detection of Resource-Constrained Language from Social Media using Transformer-Based Approach year [2024]	Sihab-Us-Sakib, Md. Ashraful Islam, S. Hossain	This paper presents a transformer-based model for detecting cyberbullying in Bengali, a low-resource language. The authors compiled a dataset of 2,751 social media comments from Facebook, YouTube, and Instagram and compared ML and DL models such	1. Achieved $F1 = 0.83$ and Accuracy = 82.61% 2. Handles sarcasm and contextual text effectively. 3. Suitable for resource-constrained languages.	1. Limited to Bengali language only. 2. Small dataset (2751 comments). 3. Offline model no real-time detection or deployment.

			as SVM, NB, RF, CNN, LSTM, GRU, and XLM-RoBERTa.		
2	BullStop: A Mobile App for Cyberbullying Prevention year [2020]	Semiu Salawu, Yulan He, Jo Lumsden	A mobile application integrating deep learning models such as BERT, RoBERTa, XLNet, and DistilBERT for detecting cyberbullying in real-time. Built with a microservices architecture, it connects to social media APIs for classification, deletion, and blocking of offensive content.	1. Real-world deployable mobile app. 2. Integrated with Twitter API for real-time detection and blocking. 3. Open-source and user feedback-based retraining.	1. Restricted to English Twitter data. 2. Moderate accuracy (~58%, macro F1=0.54). 3. Limited multilingual and cross-platform support.
3	Intelligent Multi-Lingual Cyber-Hate Detection in Online Social Networks: Taxonomy, Approach	Donia Gamal, Mahmoud Hamed, Ahmed Said	A comprehensive survey covering 25+ multilingual datasets and techniques for hate speech and cyberbullying detection. Proposed taxonomy for 10 hate types and reviewed	1. Provides a wide overview of multilingual detection systems. 2. Identifies global datasets and transformer models like BERT and XLM-R. 3. Useful reference for	1. Purely theoretical, no implementation or evaluation. 2. Lacks real-time or deployment testing. 3. No validation

	ches, Dataset s, and Open Challen ges year [2023]		methods like ML, DL, lexicon- based, and hybrid models.	multilingual research.	n across live environm ents.
4	Advanc ing Cyberb ullying Detecti on: A Hybrid Machin e Learnin g and Deep Learnin g Frame work for Social Media Analysi s year [2025]	Bishal Shyam Purkayas tha, Rupak Dey, Aritra Paul	Introduced a hybrid framework combining ML (Random Forest, XGBoost, AdaBoost) and DL (BiLSTM, BiGRU, BERT) models for social media cyberbullyin g detection. Trained using balanced datasets from Mandalay and Kaggle.	1. Achieved 91.36% accuracy and 91.24% F1- score 2. Demonstrate d interpretabili ty using SHAP/LIME 3. Balanced hybrid ML– DL approach improves detection accuracy.	1. Focused only on English datasets. 2. Computa tionally expensiv e and heavy model. 3. No multiling ual or real-time capabilit y.

2.2 Existing Work

Over the past few years, there has been a proliferation of artificial intelligence-based systems and apps to identify and combat online harassment and cyberbullying. These solutions currently in use make use of high-order Natural Language Processing (NLP) and deep learning models to detect offensive or hateful text. Although they have made a lot in enhancing online safety, they still have some weaknesses in terms of multilingual flexibility, real-time functionality, and integration of the platform.

1. Cyberbullying Prevention Mobile Applications:

Some mobile apps have been developed to detect and stop cyberbullying on the social media. Such systems use deep learning architectures like BERT and RoBERTa to identify text as abusive or harassing. They are able to analyze the comments or messages instantly and send users a notification about harmful content. Nonetheless, most of these apps are usually limited to data in the English language and one platform which makes them less applicable worldwide.

2. Language-Specific Detection Models:

Other studies have aimed at coming up with models of specific languages especially low-resource languages. Such systems employ transformer-based methods, including XLM-RoBERTa, to identify cyberbullying or hate speech in the language of the region. They are very accurate in the limits of their language, but they do not have that cross-lingual support and cannot perform in a variety of linguistic situations.

3. Survey-Based Frameworks and Taxonomy Studies:

Detailed reviews and research have identified different forms of cyberbullying and hate speech and retrieved multilingual datasets to train detection models. These surveys can be helpful in terms of understanding the model architectures, linguistic diversity, and issues related to data imbalance. They are however hypothetical and lack practical implementation structures and real time implementation strategies.

4. Deep Learning Frameworks:

New hybrid systems combine traditional machine learning algorithms with deep learning models to improve detection accuracy. The frameworks are capable of capturing contextual and semantic meaning in text in social media and attaining high performance on benchmark datasets. Although they are accurate, they are expensive to compute, and can only work in offline or English-only mode, which is not as scalable as multilingual and real-time detection requires.

2.3 Requirements Analysis

Table 2.2: Requirements

Category	Requirement Description
Frontend	Browser Extension built using HTML, CSS, and JavaScript for detecting and blurring toxic comments. Displays alerts (Toxic comment detected).
Responsiveness	Works seamlessly on desktop browsers. Can be extended for mobile browsers in future versions.
Backend	Flask API hosted on Hugging Face Spaces; handles comment text, calls model, and returns prediction.
Database	SQLite to store data for users, comments admins.
AI/NLP Model	Uses Transformer-based models like Multilingual XLM-RoBERTa for toxic content detection in multiple languages.
APIs Used	Flask REST API + Hugging Face Interface API.
Platform Integration	Initially designed for Instagram comment sections. Can be expanded to YouTube, Facebook, or Twitter by modifying DOM selectors and URLs.
Server Requirement	Hugging Face Space hosting (HTTPS-secured, no local setup needed)
User Types	General Users: View filtered comments , Admins: Manage backend, monitor predictions, retrain model
Account Features	No login needed for extension users; admin access for backend management and retraining module.
Real-Time Detection	Scans comments every 3000 ms to detect new ones, sends them to backend and applies blur immediately if toxic.
Data Storage	Stores comment text, prediction result, detection time and author ID
Security	All communication uses HTTPS via Hugging Face (Cloudflare SSL).
Performance	Real-time operation with <1 sec latency per comment.

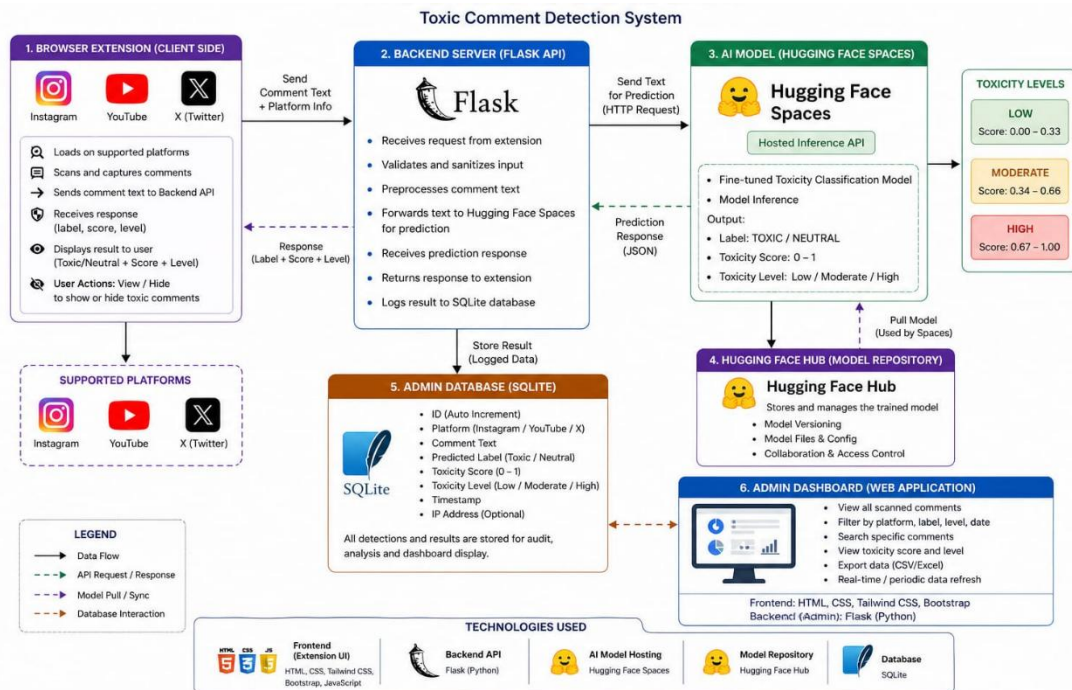
CHAPTER 3

SYSTEM DESIGN

3. System Design

3.1 Architectural Diagram

The proposed system follows a **layered architecture** to ensure modularity, scalability, and efficient processing of facial data for attendance marking. Each layer performs a specific function, contributing to the overall workflow of the system.



3.1 Architectural Diagram

Following is the detailed explanation of an architectural diagram

Next is an explanation of an architectural diagram.

1. User - Browse Extension (Frontend)

Users:

The user views social media or other websites using a browser extension (frontend). The extension constantly scans the comments or text typed in by the users. It captures when a toxic remark is spotted in real-time.

Browser Extension: Just like the Publish comment and Text input monitor, it monitors incoming comments or text entries to the webpages. It automatically strips off visible comments with functions (e.g., `scanComments()`). It forwards every comment to the backend where it is analyzed.

2. Frontend layer / Web Extension.

Built using:

CSS and JavaScript, HTML.

Gives the user interface to present toxicity results and alerts.

Functions:

- Records user comments and posts them to the API server.
- Picks up the model prediction (Toxic, Neutral).
- Shows the result of classification next to the comment.
- Provides simple and interactive interface without performance impact on browsing.

Delivers:

- Real-time comment scanning
- Hazard warnings and lights:
- Comment analysis logs
- Seamless background detection

3. Backend System (Flask-Hosted on Hugging Face Spaces)

Flask Server:

- Serves as the main back end that received all model requests and API requests.
- Gets the comment text on the browser extender over HTTPS.
- Our 90-100k words Processing: Performs preprocessing steps: tokenization, normalization, and language detection.
- Passes the text to the Hugging Face toxicity thesaurus (e.g., XLM-RoBERTa) to get a toxicity classification.
- Predicts and displays model results in firefox extension in real-time in the browser.
- Stored on Hugging Face Spaces, which offers automatic SSL certification and a secure public endpoint, giving an API.
- Eliminates any local server configuration, which ensures numerous layers of communication between the extension and backend are communicated effortlessly.

4. Database (SQLite):

Stores:

- User logs (log comment, prediction label)
- The Data references with which the model is trained.
- Serves as the hub or central data repository of performance tracking, retraining and analysis

5. NLP Preprocessing Module

- Preprocesses text for model inference.
- Strips punctuation, special characters and stop words.

- Splits the text into words or subwords.
- Identifies language and passes the text to the right model or multilingual embedding.
- Encodes the cleaned text into vectors for the model.
- Ensures that:
 - Clean, consistent data is fed to the model.
 - Multilingual comments are handled efficiently.

6. AI Model (Toxicity Detection Engine)

Built using:

- Pretrained natural language processing (NLP) models (e.g., XLM-RoBERTa).
- Trained on multilingual toxic and non-toxic data.
- Performs:
 - Embedding feature extraction.

7. API Communication Layer

- Manages communication between the frontend and backend.
- The extension makes a POST request (e.g., /predict) to the Flask API.
- The API passes the text to the XLM-R model (textdetox/xlmr-large-toxicity-classifier).
- The model returns a JSON response which is parsed by the frontend with 'await response.json()'
- e.g. { "label": "TOXIC" }
- Ensures:
 - Fast, lightweight communication.
 - Instant feedback in milliseconds.

8. Database & Logging System

Records all:

- Comments processed and their prediction outcomes.
- Data and feedback for model retraining.

Benefits:

- Allows monitoring and analytics.
- Bases information for model refraining or re-training.

9. Flow Summary

- User browses the web via the browser extension.
- The extension scans web pages for new or visible comments.
- The comment is then forwarded to the backend API for processing via POST request.
- The backend cleans the text (tokenisation, normalisation and language detection) and sends the preprocessed text to the XLM-R AI model.
- The AI makes a prediction and labels it as Toxic or Neutral. The backend sends the prediction in JSON format

- The extension receives the response and visually marks toxic comments (blurs comment and displays a warning message).
- All comments and predictions are stored in the database for performance monitoring, model evaluation and retraining.

3.2 Flowchart

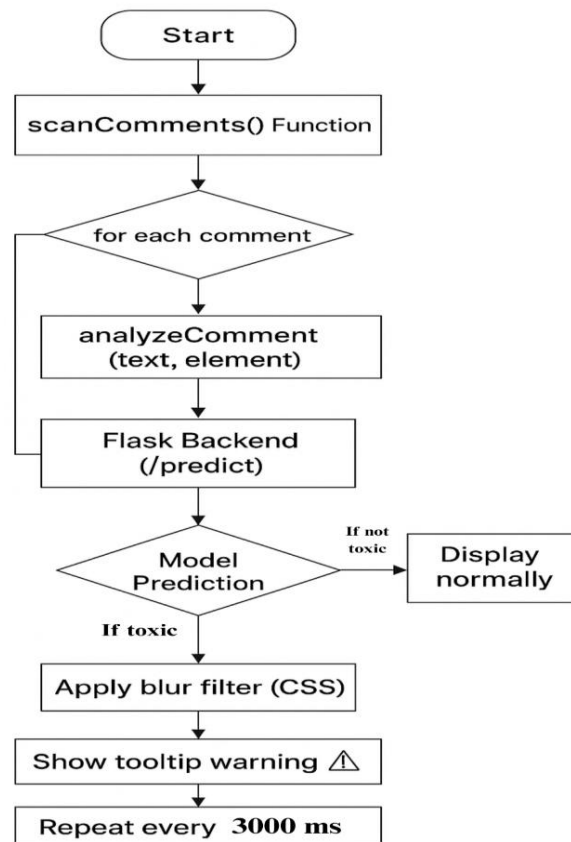


Figure 3.2 Flowchart

Comment Analysis Functionalities:

Start Process: The process starts when the user visits a social media post or a comment section.

Scan Comments (scanComments() Function): This function keeps monitoring the comments on the webpage to check for new comments.

Analyze Comment (analyzeComment() Function): For each comment, both the text and the original HTML element are sent for toxicity analysis.

Flask Backend (/predict): The comment text is forwarded to the Flask backend server, which is running the AI model.

Model Prediction: The model is used to check and classify the comment as toxic or neutral based on the parameters.

Display Result:

- If neutral, the comment is shown as it is.
- If toxic, the comment is blurred using a CSS filter to block it.
- Show Tooltip Warning: A tooltip is displayed on blurred comments to indicate that the text is possibly offensive or inappropriate.
- Repeat Process: The process is repeated every 3000 milliseconds (3 seconds) to process new comments as they are added to the page.

3.3 ER diagram for database

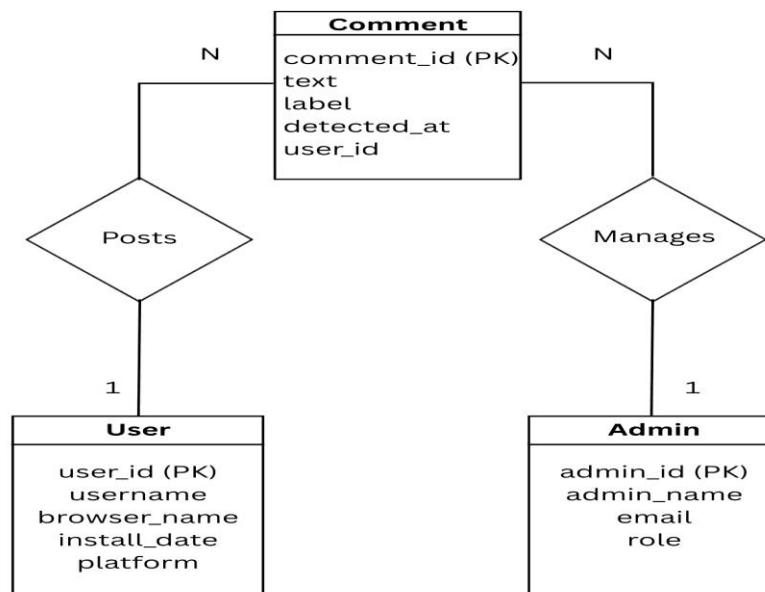


Figure 3.3 Entity Relationship (ER) Diagram

1. User

Fields: user_id (PK), username, browser_name, install_date, platform

Description: This is an end-user of the system (e.g., via a browser extension). One user can make many comments which are then analyzed for content or sentiment.

Dependency & Relation: Many Comments belong to a User. A User can have multiple Comments. User 1:N Comment

2. Comment

Attributes: comment_id (PK), text, label, detected_at, user_id (FK)

Description: Represents a user comment. The comment is classified (toxic).

Dependency & Relation: One User can create one or more Comments. A User can make multiple Comments. User 1:N Comment

Admin

Fields: admin_id (PK), admin_name, email, role.

Description: An administrator that can view, manage or moderate comments.

Dependency & Relation: An Admin can view many Comments. A Comment can be only managed by one Admin. Admin 1:N Comment.

System Workflow:

1. User performs actions on the system (e.g., posts or reads comments).
2. Comments created by the User are saved with label and detected_at information.
3. Admins can view and/or manage multiple comments, such as content moderation or validating analysis outcomes.
4. Foreign keys are used to maintain relationships (user_id is Comment and potentially admin_id in Comment).

3.4 DFD Diagram (Level 1)

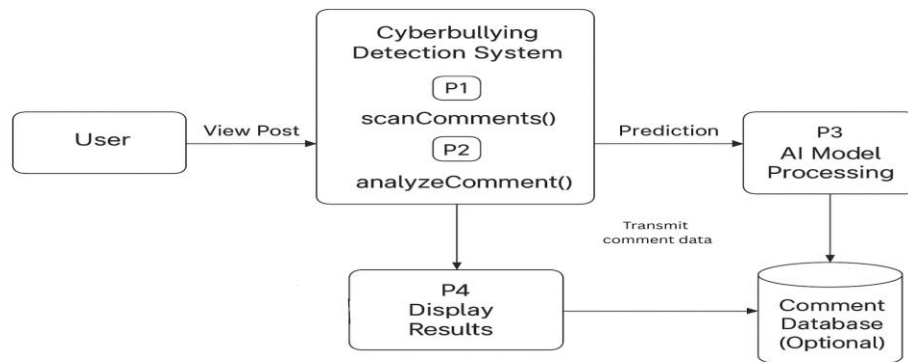


Figure 3.4 Data Flow Diagram (DFD)

The DFD above shows the flow of information between the various parts of the AI Cyberbullying Detection System. It displays the interaction between the user, process and the database that facilitates the training and moderation of the model.

3.5 Execution Flow

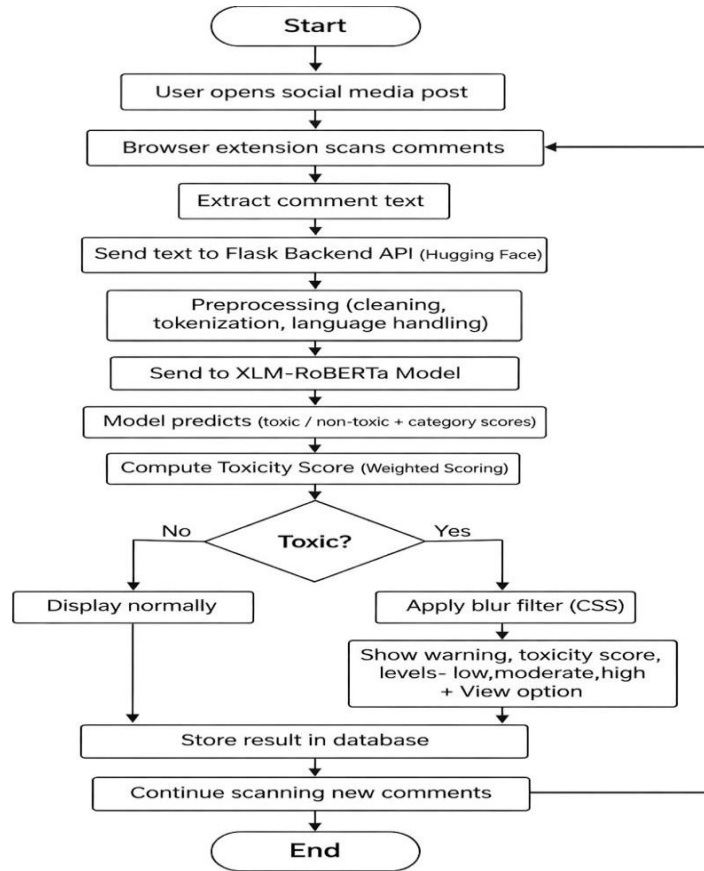


Fig 3.5 Execution Flow Diagram

The above process diagram illustrates the order in which the steps are executed for real-time classification and removal of toxic comments through the browser extension and Flask-powered system running on Hugging Face Spaces. A multilingual toxicity detection model, XLM-RoBERTa, is used to detect toxic comments, and weighted scores are used to calculate toxicity levels.

1. Start

The system starts when the user loads a social media post in the browser. The browser extension is triggered, and background scripts start scanning for visible comments.

2. User Opens Social Media Post

Once a user opens any post, the extension is triggered and the webpage is searched for comments in the post's comment section.

3. Extension Scans for Comments

The extension calls the `scanComments()` function that runs every 3 seconds (`setInterval 3000ms`). It gathers all comments that are currently on the page in the DOM, filters out new or unscored comments, and passes them to the toxicity analysis.

4. Extract Comment Text

The text of the comment is taken from the DOM element. The comment text and the DOM element are sent to the `analyzeComment()` function for further analysis.

5. Post Text to Flask API (Hugging Face Spaces)

The text of each comment is transmitted in a JSON request to the Flask API endpoint (`/predict`) via Hugging Face Spaces. This eliminates local hosting and CORS problems by enabling HTTPS.

6. Preprocessing

The Flask app preprocesses comments before feeding them to the model. This involves cleaning (stripping noise and special characters), tokenization (breaking text into model-ready tokens) and language processing (to support multilingual comments).

7. XLM-RoBERTa Model Inference

The cleaned text is sent to the XLM-RoBERTa model, a multilingual NLP transformer model. The model will assess the input text and make predictions whether the comment is toxic or non-toxic, with category-level scores.

8. Model Predicts (Toxic / Non-Toxic + scores)

The model provides a prediction (toxic or non-toxic) along with scores for different toxicity categories (hate, obscenity, insult, threat). These are then subjected to weighted scoring in the next step.

9. Compute Toxicity Score (Weighted Scoring)

The model's category scores are used to compute a final toxicity score using a weighted scoring system. We then obtain an overall toxicity rating that helps to categorise the comment into one of three groups: low, moderate or high.

10. Toxic (Decision Branch)

Depending on the toxicity score, there are two options:

No (Non-Toxic): The comment is not toxic and is rendered in the browser as usual.

Yes (Toxic): The comment is tagged and the following moderation measures are taken.

11. Apply Blur Filter (CSS)

If the comment is predicted as toxic, the extension applies a CSS blur filter (e.g., `element.style.filter = "blur(5px)"`) to the DOM to blur the comment text, ensuring that it can't be directly read but still leaves the comment intact on the page.

12. Show Warning Tooltip

A tooltip displays over the blurred comment, with the toxicity score and the toxicity level (low, moderate, or high). It also includes a "View anyway" button to allow the user to make a deliberate decision to view the content.

13. Store Result in Database

The result of the classification is logged in the database for each comment, regardless of the classification result. This is the comment text, prediction, toxicity score, and the date and time, for logging, auditing and to improve the predictive model.

14. Keep Processing New Comments

Moderation continues in a loop. The extension continues to invoke `scanComments()` using `setInterval` (every 3000ms) to check for new comments. The loop returns to the scanning stage, providing an automated real-time moderation loop.

15. End

This ends when scanning ceases (e.g., the user leaves the post, or the extension is turned off). The results are stored in the database.

CHAPTER 4

IMPLEMENTATION DETAILS

4. System Requirements

4.1 Software Requirements:

The system is designed to operate as a browser extension with an AI-powered API backend, leveraging web development and machine learning technologies.

The user interface uses JavaScript, HTML, and CSS for a lightweight and responsive design that runs in the browser.

The backend is powered by Flask (Python) for handling incoming requests, detecting toxicity with a pretrained transformer model, and real-time communication with the extension.

The AI model is deployed on Hugging Face Spaces, and the extension operates in the user's browser.

It's designed to work with popular browsers like Google Chrome and Microsoft Edge.

1. Operating System:

- Windows 10 / 11
- Linux (Ubuntu / Fedora)

2. Browser Platform

- Google Chrome (recommended)
- Edge (Chromium-based)

3. Programming Languages:

- Frontend (Extension): HTML, CSS, JavaScript
- Backend (API): Python (Flask Framework)

4. Frameworks and Libraries:

- Flask – for backend API handling
- Transformers (Hugging Face) – for loading and using pre-trained toxicity detection models.
- Torch – for deep learning model inference.

5. Database

- SQLite

6. APIs and Services:

- Hugging Face Model API (textdetox/xlmr-large-toxicity-classifier) – for detecting text toxicity.
- Chrome Extension API – for modifying browser DOM and access to permissions.

7. Development Tools:

- Visual Studio Code – for editing the code.
- Google Chrome Developer Tools – to test and debug the extension.
- Python (≥ 3.9) – to run the server locally.
- Hugging Face Spaces – for hosting the model in the cloud.

8. Other Requirements:

- Internet connectivity (to connect with Hugging Face API).
- Use of JSON for communication between front-end and back-end.
- Minimum system requirements (Dual-core processor, 4 GB memory).

4.2 Solution Approach / Methodology

This project uses a client-server, API-driven architecture design that emphasises real-time efficiency and scalability by separating the front end (the browser extension) from the back end (the AI model). The project follows a modular approach, involving browser scripting, Natural Language Processing (NLP) and secure APIs.

1. System Architecture Design

The system is based on a Two-Tier Architecture: a minimal Frontend Extension and a powerful AI Backend API.

- Design Basis: A browser extension platform using JavaScript (for DOM manipulation), CSS and Manifest V3 format.
- Backend Services: Python with Flask framework for hosting the Hugging Face Transformer model (xlmr-large-toxicity-classifier) to support efficient multilingual natural language processing (NLP).
- Data-Flow Relationship: The architecture is driven by the processing of text: Comment Text (Frontend) → REST API Call (Backend) → Toxicity Label (Backend) → UI Action (Frontend).

Table 4.1: System Architecture: Key Modules and Technologies

Module	Technology	Primary Function
Extension Configuration Module	manifest.json	Specifies the permissions, resources and, importantly, the host matching for the target system (Instagram: https://www.instagram.com/).
Scraping & Communication Module	content.js (JavaScript)	Scrapes the text of comments from the Instagram Document Object Model (DOM) in real time and sends the asynchronous (non-blocking) POST request to the AI API.
AI Core Module	Python/Flask (app.py)	Loads and runs the complex Multilingual Toxicity Classification Model and returns the classification (toxic/non-toxic).
Mitigation Module	content.js (JavaScript)	Performs a local action (blurring the text) on the UI based on AI prediction.

2. API Integration

The system relies on a single, custom-built API for its core intelligence.

- **Toxicity Prediction API (Custom Endpoint):** The extension sends a **POST request** to the remote AI server (e.g., <https://suspiciouspotato100-ToxiShield.hf.space/predict>) with the comment text in the JSON payload of the request.
- **Model Integration:** The server uses the **Hugging Face Transformers** library to load the pre-trained NLP model, making sure that we are using the most advanced model with multilingual support.

3. Toxicity Evaluation

Unlike traditional binary classification (toxic / non-toxic), The system now categorises comments into three levels of toxicity, rather than just toxic / non-toxic:

- Low Toxicity
- Medium Toxicity
- High Toxicity

This multi-level classification is achieved using a pretrained transformer-based model fine-tuned on a labeled dataset containing different toxicity intensities.

Based on the predicted level:

- Low → Mild warning
- Medium → Blur with warning
- High → Strong blur + restricted visibility

This makes it easier to interpret and allows users to better understand the impact of harmful content.

4. Model Retraining using Custom Dataset and Standard Mapping

To improve the accuracy and real-world performance of the system, the base multilingual model was further fine-tuned using a custom dataset created specifically for this project. Instead of relying solely on pretrained knowledge, we aligned our dataset with the widely accepted **Jigsaw toxicity classification standard**, which defines multiple categories of harmful content such as toxic, severe toxicity, insult, threat, and identity-based hate.

Each comment in our dataset was mapped to these standard categories to ensure consistency and reduce subjectivity. Based on these labels, a toxicity score was computed using a weighted scoring mechanism. More severe categories were assigned higher importance, ensuring that strongly harmful content contributes more to the final score.

The **scoring formula** used is:

$$\text{Toxicity Score} = 0.30 \times \text{toxic} + 0.20 \times \text{severe_toxic} + 0.15 \times \text{obscene} + 0.15 \times \text{threat} + 0.10 \times \text{insult} + 0.10 \times \text{identity_hate}$$

This score lies between 0 and 1 and represents the intensity of toxicity. The score is then interpreted into levels:

- **0 – 0.3 → Low**
- **0.3 – 0.7 → Moderate**
- **0.7 – 1 → High**

This approach ensures that the model not only detects toxicity but also understands its severity. During retraining, the model learns patterns from both the pretrained dataset and our custom dataset. The input text is tokenized and converted into numerical embeddings, which are then compared with learned patterns. Based on similarity and contextual understanding, the model generates a probability score representing toxicity.

This method aligns with standard machine learning practices where models output probabilities instead of fixed decisions, making the system more flexible and interpretable.

The model was further **fine-tuned** on a **custom dataset** enriched with Hinglish, Marathi, slang variations, masked words (using symbols like ‘*’), and **edge-case** comments that were previously misclassified, thereby improving its robustness and real-world detection capability

1. Mitigation Mechanism (Real-Time Filtering)

The mitigation mechanism in ToxiShield aims to filter out harmful content from clients in real-time using a pretrained model. This approach keeps the user safe, with transparency and user control.

Toxicity Detection Using Pretrained Model:

The system utilizes a **pretrained toxicity classification model** trained on labeled datasets to categorize user content into three toxicity levels:

- Low Toxicity
- Medium Toxicity
- High Toxicity

Comments are processed locally, and a toxicity label is assigned based on prediction.

Dynamic Content Filtering (Blurring): The system dynamically blurs content.

- Neutral: Content is displayed normally
- Low/Medium/High Toxicity: Content is blurred using a CSS filter
- A warning message is attached to the element using a tooltip
- To prevent censorship and provide users with more control, a "View" button is provided alongside blurred content.

Appears for low, medium and high toxicity content.

Allows users to temporarily view hidden content.

2. Testing & Validation

The test is concerned with the speed, accuracy and deployment of the two main modules.

- **Functional Testing:** Tested the primary functionality the ability to scrape text from the Instagram DOM, the success of API calls/responses, and the successful application of the CSS blur filter.
- **AI Model Testing:** Tested the model accuracy on a standard dataset and how long the API took to respond (500ms or less for real-time prediction).
- **API Testing:** Verified the /predict endpoint of the remote Flask server is secure, reliable, and can handle multiple clients.

3. Future Enhancements

The Instagram prototype is a stepping-stone towards a cross-platform service, which will be the subject of the project next semester:

- **Cross-Platform Extension:** Modifying the manifest.json to include host_permissions for other popular social media applications (e.g., X, Instagram, YouTube comments).
- **Adaptive Scraping:** Building a flexible content.js script that can switch between DOM selectors to select the comment elements on different platforms.
- **User Settings Dashboard:** Creating a persistent user interface to allow them to choose the mitigation action (e.g., blur/hide, or display a warning).
- **Advanced AI Features:** Implementing AI to classify the toxicity type (e.g., hate speech, harassment, profanity).

4.3 Algorithm and Core Functionality

The real-time and intelligence of the system is encapsulated in three key code components: the AI pipeline, API and the front-end comments mitigation logic.

1. Extension Configuration (manifest.json)

The manifest.json file is the manifest of the ToxiShield browser extension, which contains metadata, permissions and behaviours. Importantly, it specifies two things:

- **Host Permissions:** It provides access to the target web page (<https://www.instagram.com/>*) to retrieve comments and the distant web server hosting the AI API (https://*.hf.space/*) to make requests.
- **Content Script Injection:** It ensures the content.js file is automatically injected and executed on all matching pages ([instagram.com/](https://www.instagram.com/)*) for the application logic to interact with the dynamic website content.

The code snippet below is the configuration for the current prototype (Manifest V3):

manifest.json

```
{
  "manifest_version": 3,
  "name": "ToxiShield",
  "version": "1.0",
  "permissions": ["scripting", "activeTab"],
  "host_permissions": [
    "https://*.hf.space/*",
    "http://127.0.0.1:5001/*"
  ],
  "content_scripts": [
    {
      "matches": [
        "https://www.instagram.com/*",
        "https://twitter.com/*",
        "https://x.com/*",
        "https://www.youtube.com/*"
      ],
      "js": ["content.js"]
    }
  ]
}
```

2. AI Backend API Setup (app.py - Python/Flask)

This is the "brain" of the system on the server side. This script simply provides a low-latency REST API to predict toxicity, allowing the heavy lifting to be done on the server, rather than in the browser extension. This sets up the AI model and the /predict API call. It loads a Multilingual Classifier model, which will enable toxicity detection in multiple languages.

3. Frontend Detection and API Call (content.js - JavaScript)

This JavaScript function is injected into Instagram page. It is responsible for the vital function of transmitting the comment text to the remote AI.

4. Mitigation Algorithm (UI Action Logic)

The last action is the mitigation algorithm, which responds immediately to a positive toxicity tag from the AI.

Table 4.2: Mitigation Algorithm Explanation

Condition	Action (Code Snippet)	Purpose
If label is 'toxic'	<code>element.style.filter = "blur(6px)";</code>	Applies a blur filter to the text element to make it impossible for the user to see the toxic content, while maintaining the social media page's visual structure.
Metadata Update	<code>element.title = "Toxic comment detected";</code>	Adds an attribute (hover-over text) to inform the user why the text was blurred.

5. Model Performance Improvement

To evaluate the effectiveness of retraining, we compared the model's **performance before and after fine-tuning** on our custom dataset. Initially, the model achieved an accuracy of approximately **78%**, which improved to around **87% after retraining**. This improvement demonstrates that incorporating domain-specific and multilingual data significantly enhances the model's ability to correctly classify toxic and non-toxic comments. The retrained model shows better contextual understanding, especially for mixed-language and informal social media text.

4.4 Outputs and Results

This section uses illustrative evidence to validate the entire system, the effectiveness of the AI model and the extension's ability to mitigate for the system in real-time on the target platform.

Live Extension Output: Successful Mitigation

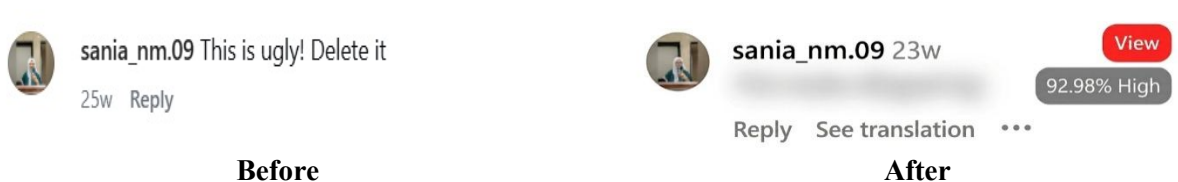


Figure 4.2: Toxic Comment filtering on Instagram

In addition to Instagram, the system was tested on **YouTube comment sections** to evaluate its performance across different content platforms. The browser extension successfully scanned and analyzed comments in real time, accurately identifying toxic content. The model demonstrated strong performance in handling diverse comment styles, including informal language and mixed expressions commonly found in video discussions. With the updated implementation, the system now displays both the **toxicity level (Low, Moderate, High)** and **the corresponding confidence score**. This enhancement improves interpretability and enables users to better understand the severity of harmful content while browsing.

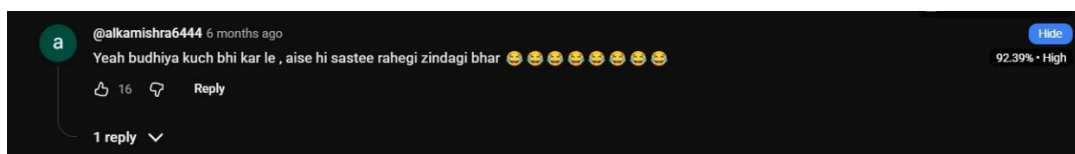


Figure 4.2: Before toxic comment filtering on Youtube

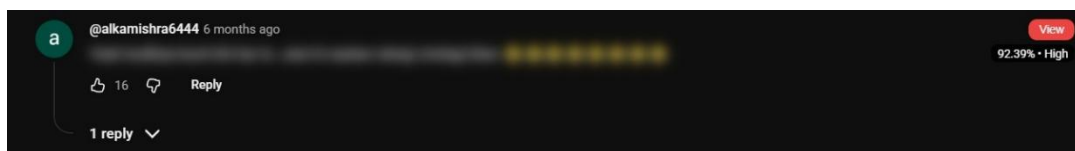


Figure 4.3: After toxic comment filtering on Youtube

The system was further evaluated on **X (Twitter)** to test its adaptability to short-form and fast-paced conversational text. The extension effectively detected toxic replies and classified them in real time, even in cases involving abbreviations, slang, and code-mixed language. The output includes both the predicted label and a detailed toxicity score, which is mapped to defined levels of severity. This ensures consistency in user experience across platforms. The **successful performance on X** highlights the robustness of the model and its ability to generalize across different types of social media interactions.

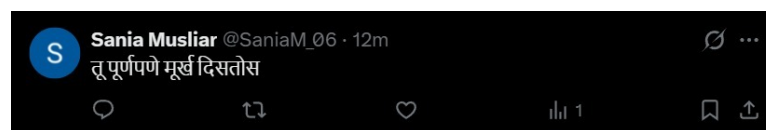


Figure 4.4: Before toxic comment filtering on X

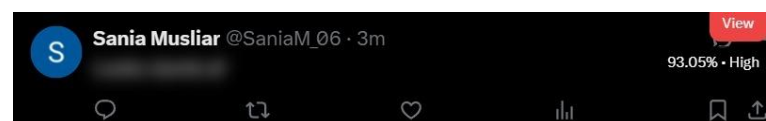


Figure 4.5: After toxic comment filtering on X

CHAPTER 5

SOFTWARE TESTING/TEST CASES

5. Software Testing and Test Cases

Software testing was conducted across three categories: functional testing of each recognition modality, security testing of the anti-spoofing module, and integration testing of the end-to-end attendance pipeline. All tests were performed on the evaluation hardware (Intel Core i5, 8 GB RAM, no GPU) using the 61-student enrolled dataset.

5.1 Unit Testing

Unit testing was performed to verify the correctness of individual modules within the system. Each component was tested independently to ensure it functions as expected

Comment Extraction Module:

The scan Comments() function was tested to ensure accurate extraction of comments from the web page DOM structure across different scenarios, including dynamically loaded content.

Text Preprocessing Module:

Preprocessing steps such as tokenization, normalization, and removal of special characters were tested to ensure consistent and clean input to the AI model.

API Communication:

The Flask API endpoint (/predict) was tested using sample inputs to validate correct request handling, response formatting, and error handling.

These tests ensured that each unit performs reliably before integration.

5.2 Integration Testing

- Integration testing was conducted to verify the seamless interaction between different components of the system.
- Validated communication between the browser extension (frontend) and Flask backend API Ensured proper data flow:
Comment → API Request → Model Prediction → Response → UI Action
- Verified that the system correctly processes multiple comments in sequence without delays or failures

This testing confirmed that all modules work together cohesively in an end-to-end environment.

5.3 Model Evaluation

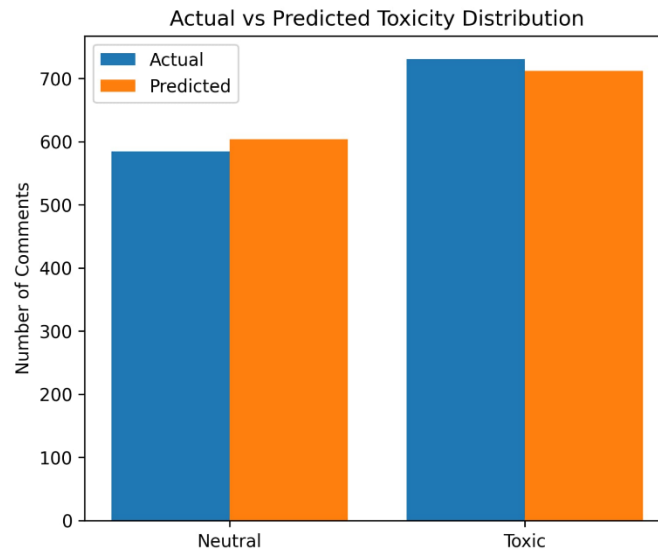


Figure 5.1: Actual vs Predicted Toxicity Distribution

The above graph compares the distribution of actual labels and model predictions for neutral and toxic comments. The close alignment between actual and predicted values indicates that the model performs reliably with minimal deviation. This demonstrates that the **retrained model** has learned meaningful patterns from the dataset and can generalize effectively on unseen data.

5.4 Real-Time Testing

Real-time testing was performed to evaluate the system's behavior in a live environment.

- The extension was deployed on social media platforms (e.g., Instagram)
- The system continuously scanned comments at regular intervals (every few seconds)
- Newly appearing comments were automatically detected and processed

Key observations:

- Fast response time (near real-time detection)
- Immediate application of blur effect on toxic content
- No noticeable lag in user browsing experience
- This confirms the system's ability to function effectively in real-time scenarios.

5.3 Real-Time Testing

Real-time testing was performed to evaluate the system's behavior in a live environment.

- The extension was deployed on social media platforms (e.g., Instagram)
- The system continuously scanned comments at regular intervals (every few seconds)
- Newly appearing comments were automatically detected and processed

Key observations:

- Fast response time (near real-time detection)
- Immediate application of blur effect on toxic content
- No noticeable lag in user browsing experience

This confirms the system's ability to function effectively in real-time scenarios

5.4 Usability Testing

Usability testing was carried out to ensure that the system is intuitive and user-friendly.

The following features were evaluated:

- **BlurEffect:**
Verified that toxic comments are visually hidden without disturbing page layout.
- **WarningMessage:**
Tooltip messages were tested to ensure users are informed about hidden content.
- **ToxicityLevels:**
The system's classification into low, medium, and high levels was checked for clarity and usefulness.
- **ViewButton:**
The "View" button was tested to ensure:
 - Smooth interaction
 - Immediate visibility toggle
 - No disruption in page functionality

CHAPTER 6

EXPERIMENTAL RESULT

6. Experimental Results

6.1 GUI Screenshots

This is the web-based admin login page for **ToxiShield**, an AI Cyberbullying Detection System, running locally at 127.0.0.1:5001 (Flask development server). The interface follows a dark-themed, two-panel layout with a red/coral accent color scheme.

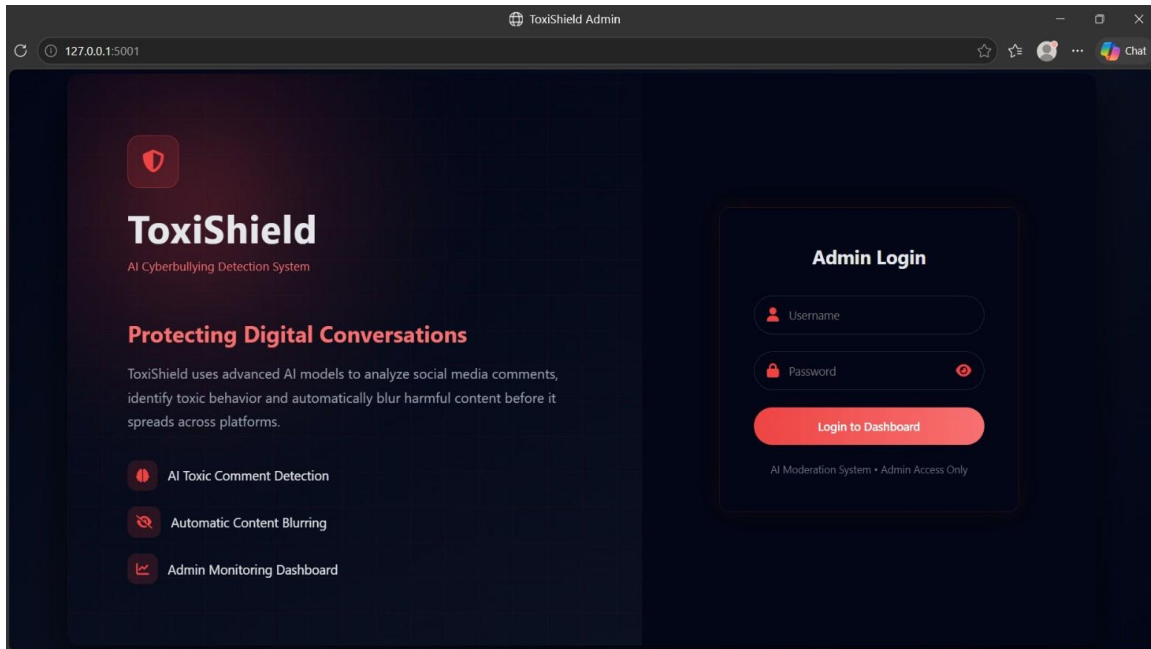


Figure 6.1: Admin login interface

Left Panel — Branding

- Red shield logo with the name "ToxiShield" and tagline "AI Cyberbullying Detection System"
- Headline: "Protecting Digital Conversations"
- Three feature highlights: AI Toxic Comment Detection, Automatic Content Blurring, and Admin Monitoring Dashboard
- Right Panel — Login Form
- Username and Password input fields (with visibility toggle)
- A coral "Login to Dashboard" button

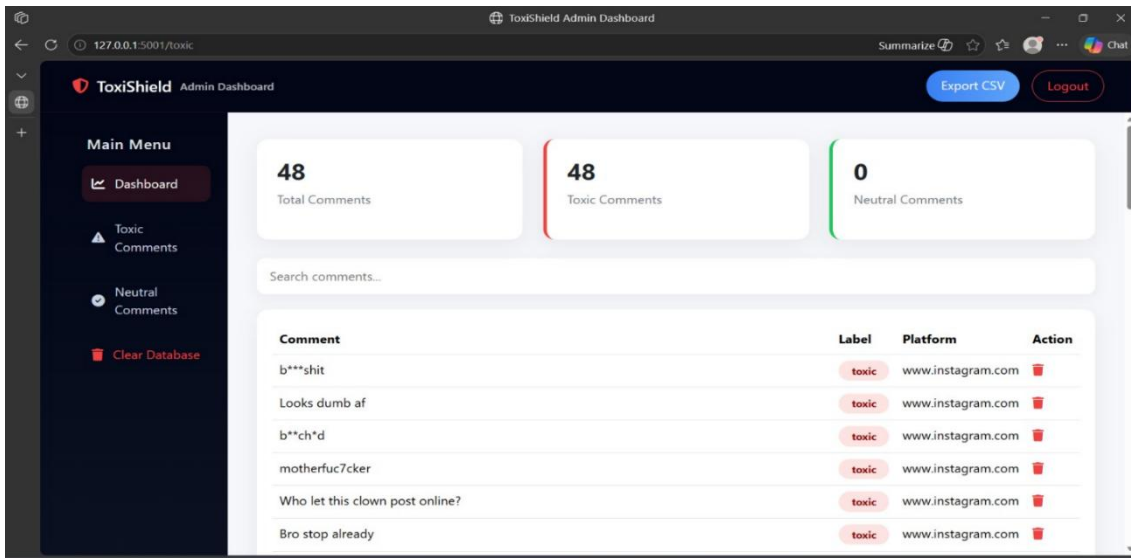


Figure 6.2: Admin dashboard showing toxic comments

The ToxiShield Admin Dashboard (at 127.0.0.1:5001/toxic) displays real-time monitoring of detected toxic comments. All 48 analyzed comments are flagged as toxic (0 neutral), sourced from www.instagram.com. The table lists each comment with its toxicity label, platform, and a delete option. Admins can search comments, export data as CSV, or clear the database from the sidebar.

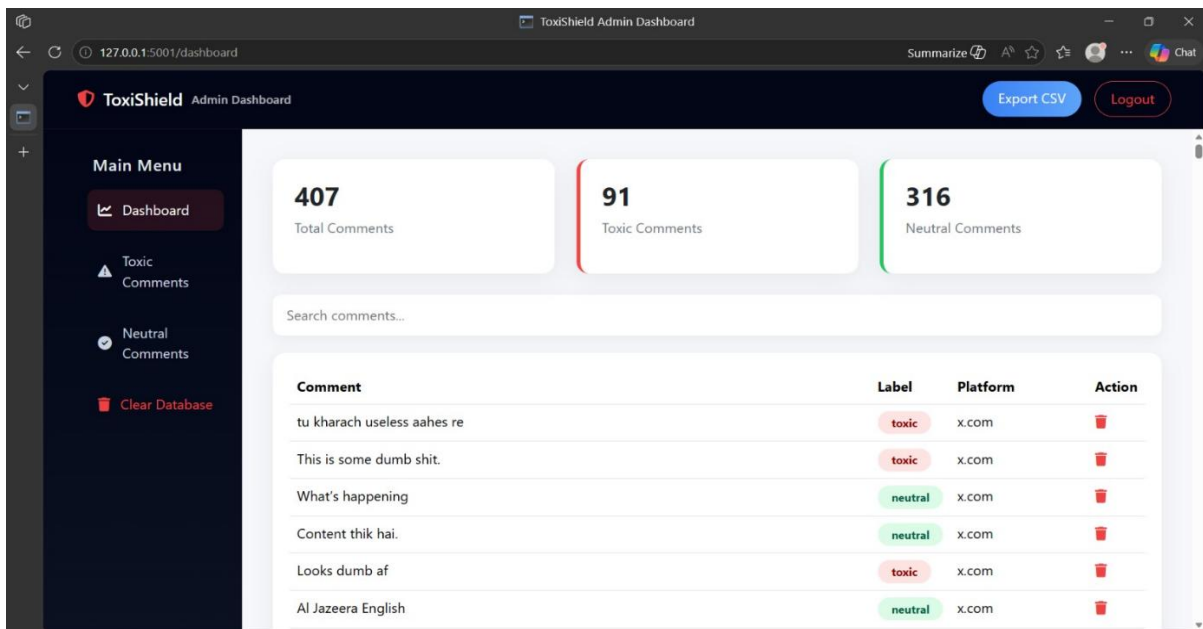


Figure 6.3: Admin Dashboard Showing Stored Comments and Predictions

6.2 Gantt chart

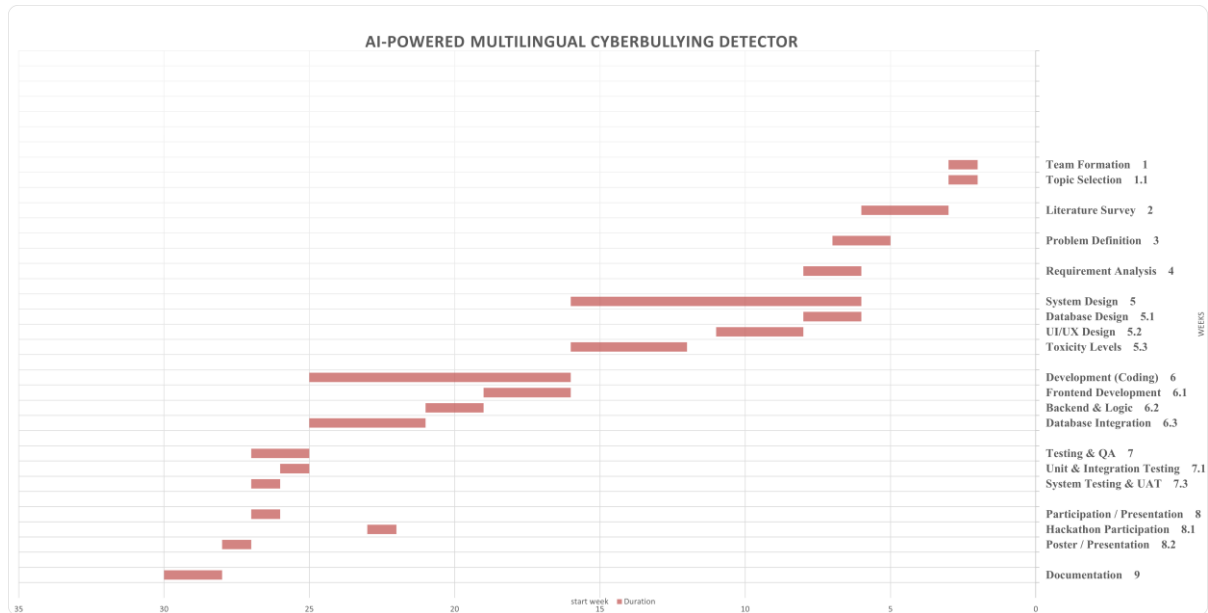


Figure 6.4: Timeline chart

This Gantt chart presents the full project schedule across approximately 35 weeks, plotted from right (week 0) to left (week 35), with each bar showing the start week and duration of every task.

The project is organized into 9 major phases:

- **Phase 1** — Team Formation (Week ~33–34): The project kicks off with team formation and topic selection, happening in the earliest weeks.
- **Phase 2** — Literature Survey (Week ~30–32): A review of existing research and prior work on cyberbullying detection and multilingual NLP.
- **Phase 3** — Problem Definition (Week ~28–30): Clearly scoping the problem statement and objectives of the system.
- **Phase 4** — Requirement Analysis (Week ~26–28): Gathering functional and non-functional requirements for the detector.
- **Phase 5** — System Design (Weeks ~18–26): The most expansive phase, broken into three sub-tasks — Database Design (5.1), UI/UX Design (5.2), and Toxicity Levels (5.3) — running in parallel across several weeks.
- **Phase 6** — Development / Coding (Weeks ~14–22): Implementation split into Frontend Development (6.1), Backend & Logic (6.2), and Database Integration (6.3), with sub-tasks staggered across the mid-project period.

- **Phase 7** — Testing & QA (Weeks ~10–14): Covers Unit & Integration Testing (7.1) and System Testing & UAT (7.3), ensuring the system functions correctly before deployment.
- **Phase 8** — Participation / Presentation (Weeks ~4–8): Includes Hackathon Participation (8.1) and Poster/Presentation preparation (8.2), toward the project's close.
- **Phase 9** — Documentation (Weeks ~1–3): Final documentation of the entire system, wrapping up just before week 0.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The AI-Powered Multilingual Cyberbullying Detection System is an effective and smart approach to combating the growing problem of cyberbullying and hate speech on social media platforms. The system uses Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to automatically process users' comments in multiple languages and detect abusive or offensive comments in real-time.

The AI-Powered Multilingual Cyberbullying Detection System is an effective and smart approach to combating the growing problem of cyberbullying and hate speech on social media platforms. The system uses Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to automatically process users' comments in multiple languages and detect abusive or offensive comments in real-time.

Through features like real-time scanning, automatic blurring of toxic comments, and user alerts, the system proactively protects users from exposure to harmful text. It helps promote positive digital interactions, increases user safety, and sets a foundation for scalable AI-based moderation system.

This project not only showcases the potential of AI for digital well-being it also opens the door to incorporating such tools into mainstream social media to foster a more positive and accepting online community.

Future Scope

The current implementation of the system is limited to detecting toxic comments in text-based posts on Instagram through a browser extension. However, the design is modular and can be extended to support many additional features in the future:

Multi-Platform Expansion:

The model can be applied to other social media platforms such as Facebook, and Reddit, by modifying the DOM selectors and API endpoints in the code.

1. Voice and Image Detection:

The model can be extended to detect toxic audio, memes or images using Computer Vision and Speech Recognition.

2. User Dashboard and Analytics:

The admin dashboard can include user reports and language statistics.

3. Offline and Edge AI Support:

If the model is run locally using ONNX Runtime or TensorFlow Lite, the system can operate off-line and be faster.

4. Contextual and Sentiment Analysis:

The system can be extended to include contextual and emotion analysis, which will help to reduce false positives and increase model interpretability.

5. Multimedia and Cross-Language Translation:

It can be applied with Google Translate API or DeepL API to support analysis of mixed language comments and improve its accuracy.

6. AI Moderation Network:

This project can grow into a general AI moderation service, which the APIs of different social media apps can join to achieve real-time automated moderation at scale.

In summary, future work on this system will focus on converting it from a prototype to a full-fledged AI moderation system, helping to create a safer and more inclusive internet..

CHAPTER 8

REFERENCES

8. References

[1] Cyberbullying Detection of Resource-Constrained Language from Social Media using Transformer-Based Approach.

Authors: Sihab-Us-Sakib et al., 2024.

[2] BullStop: A Mobile App for Cyberbullying Prevention.

Authors: Semiu Salawu et al., 2020.

[3] Intelligent Multi-Lingual Cyber-Hate Detection in Online Social Networks.

Authors: Donia Gamal et al., 2023.

[4] Advancing Cyberbullying Detection: A Hybrid Machine Learning and Deep Learning Framework.

Authors: Bishal Shyam Purkayastha et al., 2025.

ACKNOWLEDGEMENT

The making of the project “Mini Project – 1A AI - Powered Multilingual Cyberbullying Detector” involves the contribution of many people. We would like to convey our sincere thanks to Dr. S.M. Khot, principal Fr. C Rodrigues Institute of Technology, Vashi for giving us the opportunity to showcase our skills and providing us with the necessary resources. We would also like to convey our heartfelt gratitude to the Head of Department of Information Technology, Dr. Shubhangi Vaikole for her constant support and motivation. We express deep gratitude to our project guide and mentor Dr. Trupti Lotlikar for her constant motivation to think out of the box and immense contribution throughout to this project. Last but not the least, we convey our heartfelt thanks to the major project coordinator, Prof Mrs. Neelima A Kulkarni for supporting and guiding us throughout the process. We also extend our heartfelt thanks to our families and well-wishers.

Yours sincerely,

Sania Musliar

Navya Roshni

Vishakha Talele

Tanaya Deshmukh



Agnel Charities

Fr. C. Rodrigues Institute of Technology, Vashi

(An Autonomous Institute & Permanently Affiliated to University of Mumbai)

Department of Information Technology

APPENDIX A: CODE SAMPLE

1. Comment Extraction (Frontend)

```
const comments = document.querySelectorAll(".comment-class");
comments.forEach(comment=>{analyzeComment(comment.innerText,
comment)});
```

- Extracts comments from the webpage DOM
- Sends each comment for toxicity analysis

2. API Call to Backend

```
Constresponse=awaitfetch("https://yourapi/predict",{method:"POST",
body:JSON.stringify({text:comment}),});
```

- Sends comment text to Flask API
- Uses asynchronous request for real-time processing

3. Flask API Endpoint

```
@app.route('/predict', methods=['POST'])
def predict():
    text = request.json['text']
    result = model(text)
    return jsonify(result)
```

- Receives comment text
- Sends it to AI model
- Returns prediction as JSON

4. Tokenization and Model Input

```
inputs=tokenizer(text,return_tensors="pt")
outputs = model(**inputs)
```

- Converts text into tokens
- Prepares input for transformer model

5. Toxicity Scoring Logic

Toxicity Score = $0.30 \times \text{toxic} + 0.20 \times \text{severe_toxic} + 0.15 \times \text{obscene} + 0.15 \times \text{threat} + 0.10 \times \text{insult} + 0.10 \times \text{identity_hate}$

- Computes overall toxicity score
- Combines multiple categories into one value

6. Decision Logic

if score > 0:

 label = "TOXIC"

else:

 label = "NEUTRAL"

- Converts score into final label

7. UI Mitigation (Blur)

element.style.filter = "blur(6px)";

- Hides toxic content
- Protects user from harmful text

8. Tooltip Display

element.title = "Toxic comment detected";

- Provides explanation to user
- Improves transparency

APPENDIX B:

Project competition participation/winner's certificates/Certificates of papers presented at conferences/ Papers published in conference/journal

WINNER CERTIFICATE:



PARTICIPATION CERTIFICATES:



